COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT723 - DEVOPS LABORATORY

PROJECT

GAME RENTAL TRACKING SYSTEM

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “GAME RENTAL TRACKING SYSTEM” is the bonafide work of the student team completed during the academic year 2025–2026 for the project presentation of 23IT723 – DEVOPS LABORATORY.

This project was carried out in accordance with the requirements and standards of the course and demonstrates practical implementation of Git, Jenkins, Docker, testing, and deployment automation.

The work was reviewed and approved by the project review committee and is submitted as a part of the final mini-project evaluation.

Review Members:

Dr. M. Sangeetha

Dr. E. Arul

FINAL MINI PROJECT – EVALUATION SCHEME

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Project scope, requirement analysis, module design, architecture and DevOps workflow planning.	10
2	Git Repository & Version Control	Repository setup, meaningful commits, README, .gitignore, branches, merge history and remote hosting.	15
3	Branching, Merging & Collaboration	Feature development workflow, branch management, pull integration and issue tracking readiness.	10
4	Application Development & Build	Working application, Flask web implementation, database logic, and local execution.	15
5	Jenkins Continuous Integration	Build configuration, source checkout, pipeline execution and console evidence.	15
6	Docker / Containerization	Dockerfile, image generation, container runtime and deployment packaging.	10
7	Automation / Configuration Management	Ansible playbook, inventory configuration, and repeatable deployment automation.	5
8	Testing & Verification	Pytest test suite, application validation and troubleshooting evidence.	10
9	Documentation & Presentation	Technical report, architecture diagram, screenshots and presentation clarity.	10

TABLE OF CONTENTS

1. INTRODUCTION

2. OBJECTIVES

3. ABSTRACT

4. TECHNOLOGY STACK

5. SYSTEM ARCHITECTURE

6. MODULE 1 — USER INTERFACE & GAME CATALOG

7. MODULE 2 — RENTAL REQUEST MANAGEMENT

8. MODULE 3 — TRACKING & STATUS MONITORING

9. MODULE 4 — ADMIN DASHBOARD

10. MODULE 5 — DEVOPS AUTOMATION

11. RESULTS AND OUTPUT

12. COMMANDS USED

13. ADVANTAGES

14. LIMITATIONS

15. FUTURE ENHANCEMENTS

16. CONCLUSION

1. INTRODUCTION

The Game Rental Tracking System is a web-based application developed to simplify the process of browsing, renting, and tracking video games across various gaming platforms. The system offers a centralized interface for customers to view available games, choose rental terms, and receive a tracking ID for monitoring the request status.

The application enables administrators to review requests, approve or update workflow status, and monitor the progress of each rental order in an efficient and structured manner. The project is designed to demonstrate end-to-end DevOps practices in a realistic workflow using source control, automated testing, containerization, and deployment automation.

2. OBJECTIVES

- To develop a responsive web application for browsing and renting games across PC, PlayStation 5, Xbox Series X and Nintendo Switch platforms.

- To provide a structured rental workflow with validation, priority selection, and unique tracking IDs for each request.

- To maintain a centralized SQLite database for catalog data and rental request records.

- To implement a secure and user-friendly admin dashboard for managing request workflows and statuses.

- To integrate DevOps practices such as Git version control, Jenkins CI/CD automation, Docker containerization, pytest testing, and Ansible deployment.

- To demonstrate persistent storage and deployment automation in a realistic laboratory environment.

3. ABSTRACT

The Game Rental Tracking System is a DevOps laboratory project that combines a Flask web application with a SQLite database, automated testing, Docker packaging, and deployment automation. The system allows users to browse a game catalog, submit rental requests with customized durations and priorities, and monitor the status of each request using an assigned tracking ID.

Administrators can manage all incoming rental requests from a dedicated dashboard and change statuses from Pending to Approved, Dispatched, and Completed. The project further demonstrates the practical use of continuous integration and deployment using Jenkins and Ansible, ensuring that code is tested and deployed in a repeatable manner. The implementation also covers persistent storage using Docker volumes so that the SQLite database remains available across container recreation and deployment cycles.

4. TECHNOLOGY STACK

Technology	Version	Purpose
Python	3.11	Backend development and application logic
Flask	3.0.3	Web application framework
SQLite	3.x	Application database for games and rental requests
HTML5	HTML5	User interface structure
CSS3	CSS3	Responsive styling and layout
JavaScript	ES6+	Frontend interactivity
Pytest	8.2.2	Automated unit and route-level testing
Docker	Latest / project-defined	Containerization and runtime packaging
Jenkins	Pipeline-based	Continuous integration and deployment automation
Ansible	Automated playbook	Configuration and deployment orchestration
Git	Versioned repository	Source control management

5. SYSTEM ARCHITECTURE

The application architecture follows a three-tier design in which the user interacts with the Flask frontend, requests are validated in the backend logic, and database updates are stored in SQLite. The project also includes a DevOps layer where source control, testing, containerization, and automated deployment are integrated into a single workflow.

Developer -> GitHub Repository -> Jenkins Pipeline -> Docker Build -> pytest Validation -> Ansible Deployment -> Running Docker Container -> SQLite Database and Persistent Volume

Figure 5.1: System Architecture Diagram Placeholder

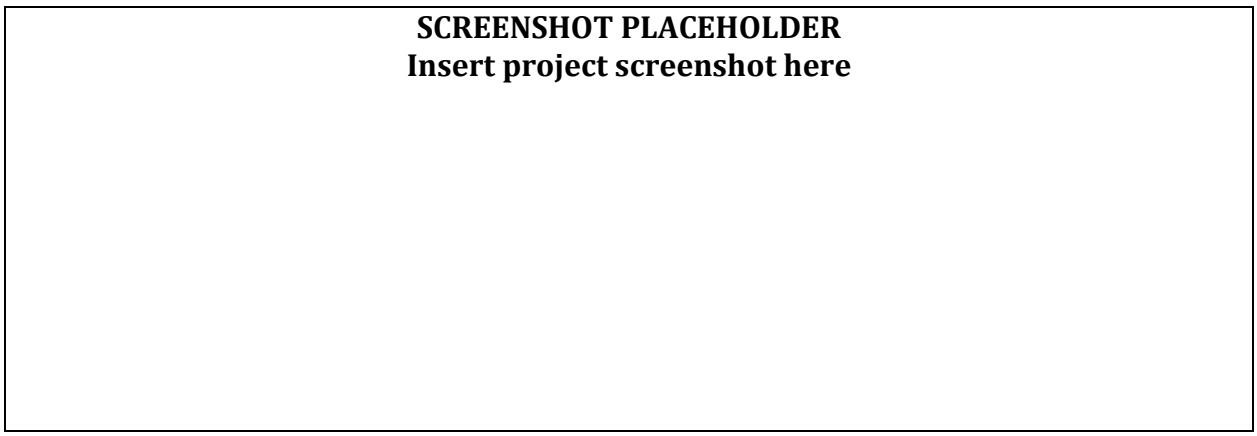


Figure: Figure 5.1: System Architecture Diagram Placeholder

6. MODULE 1 — USER INTERFACE & GAME CATALOG

This module provides the home page and catalog interface for users to browse available game titles. It displays the game list across multiple platforms and allows users to quickly read the details of each title before making a rental decision.

Main features include browsing, platform-aware catalog display, featured game cards, and direct links to the rental form. The frontend is developed using HTML5, CSS3, and JavaScript to ensure responsiveness and a clean browsing experience.

Figure 6.1: Home Page and Game Catalog Screenshot Placeholder

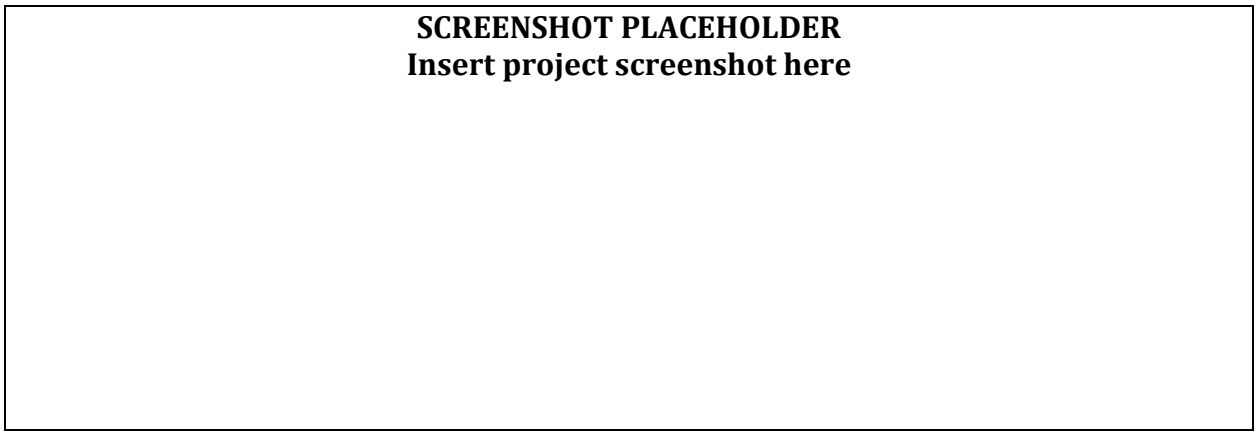


Figure: Figure 6.1: Home Page and Game Catalog Screenshot Placeholder

7. MODULE 2 — RENTAL REQUEST MANAGEMENT

The rental module handles input validation and request submission. Users provide personal details, select a game title, choose a platform, specify rental duration, and set the urgency level for the request. The backend validates the input and stores the order in the SQLite database.

The rental form includes validation for empty fields, invalid durations, and unsupported request values, ensuring that the stored data remains consistent and secure. The request is assigned a unique rental ID and forwarded to the tracking page for confirmation.

Figure 7.1: Rental Form Submission Screenshot Placeholder

SCREENSHOT PLACEHOLDER
Insert project screenshot here

Figure: Figure 7.1: Rental Form Submission Screenshot Placeholder

8. MODULE 3 — TRACKING & STATUS MONITORING

This module provides the live tracking experience for customers. After request submission, the application redirects the user to the tracking page, where the assigned request ID can be used to see the current order status. The tracking logic queries the SQLite database and displays the request details in readable format.

The status lifecycle follows a clear operational path: Pending -> Approved -> Dispatched -> Completed. Each stage is updated by the administrator and reflected in the user-facing tracking view.

Figure 8.1: Rental Tracking Page Screenshot Placeholder

SCREENSHOT PLACEHOLDER
Insert project screenshot here

Figure: Figure 8.1: Rental Tracking Page Screenshot Placeholder

9. MODULE 4 — ADMIN DASHBOARD

The admin dashboard centralizes all rental requests and provides summary KPIs such as total requests, pending requests, active requests, and completed rentals. Administrators can view the detailed list of submissions and update the status of each request as the workflow progresses.

The dashboard also supports filtered views for individual status categories. It forms a critical operational component of the project, allowing the system to manage rentals in a streamlined and transparent manner.

Figure 9.1: Admin Dashboard Screenshot Placeholder

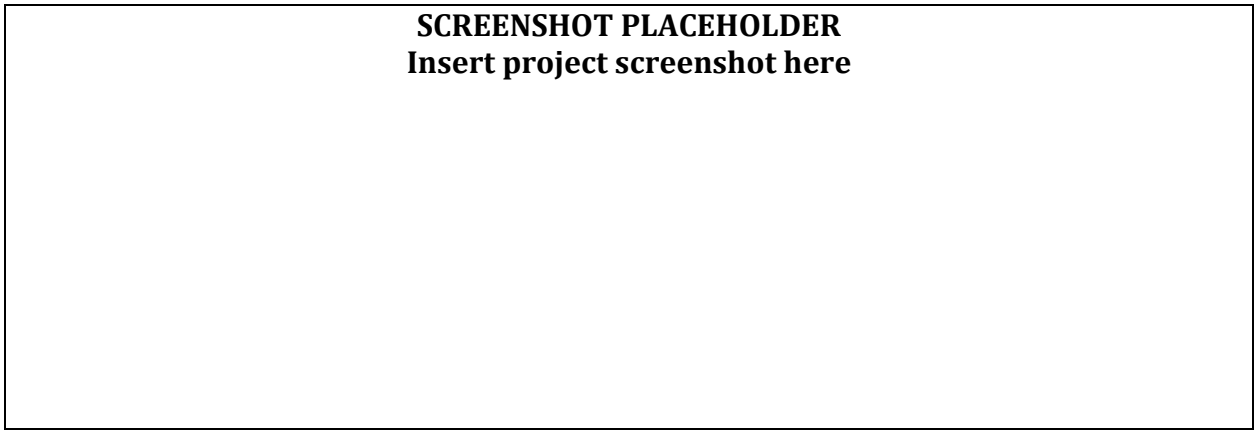


Figure: Figure 9.1: Admin Dashboard Screenshot Placeholder

10. MODULE 5 — DEVOPS AUTOMATION

The DevOps component of the project is implemented using Git, GitHub, Jenkins, Docker, Pytest, and Ansible. These tools are used in sequence to ensure that the application is built, validated, and deployed in a consistent and repeatable manner.

The Jenkins pipeline executes the Docker build step, runs the automated pytest suite, and then triggers the deployment playbook via Ansible. The Docker configuration uses a persistent volume mapped to the SQLite data directory so that rental records remain available across container lifecycle events.

Figure 10.1: Jenkins Pipeline or Deployment Output Placeholder

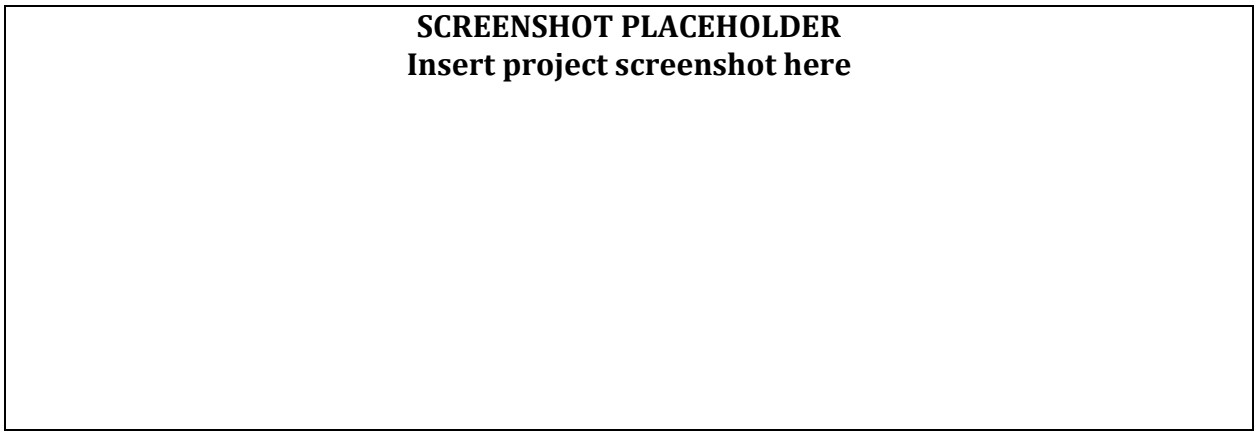


Figure: Figure 10.1: Jenkins Pipeline or Deployment Output Placeholder

11. RESULTS AND OUTPUT

The deployed application successfully loads the game catalog, allows users to submit rental requests, and provides a functional status-tracking workflow. The project demonstrates a realistic development lifecycle from application coding to end-to-end deployment using container and automation tools.

The test suite confirms the major system behaviors, including startup validation, home page readiness, rental submission, status tracking, admin dashboard access, and workflow updates. The implementation therefore satisfies the core learning outcomes of the DevOps laboratory by integrating source control, automated testing, and deployment automation into one project.

12. COMMANDS USED

python -m venv .venv

pip install -r requirements.txt

python app.py

python -m pytest test_app.py -v

docker build -t game-rental-ci .

docker run --rm game-rental-ci python -m pytest test_app.py

docker run -d --name game-rental-ansible -p 5002:5000 -v game_rental_data:/app/data --restart unless-stopped game-rental-ci

ansible-playbook -i inventory.ini game-deploy.yml

13. ADVANTAGES

- User-friendly application flow for browsing and renting games.

- Centralized and structured management of rental requests using SQLite.

- Automated validation reduces input errors and inconsistent records.

- Jenkins and Ansible enable repeatable CI/CD operations.

- Docker provides portability and environment consistency.

- Persistent storage keeps data alive across container recreation.

14. LIMITATIONS

- The project currently uses SQLite, which is suitable for demonstration but less scalable than enterprise-grade database systems.

- The UI is responsive but does not include advanced multi-role authentication and authorization.

- The application is designed primarily for a laboratory demonstration environment rather than large-scale commercial deployment.

- The system can be expanded with payment integration, inventory management, and user authentication.

15. FUTURE ENHANCEMENTS

- Add secure user authentication and role-based access for customers and administrators.

- Introduce payment flow, borrowing history, and late-return penalties.

- Integrate real-time notifications for status updates.

- Support multi-city or multi-store inventory handling with a more scalable database backend.

- Add analytics dashboards and monthly usage reports for administrators.

16. CONCLUSION

The Game Rental Tracking System successfully demonstrates how a modern web application can be developed, automated, tested, and deployed using DevOps principles. The project addresses the practical need for a digital rental workflow while highlighting the value of version control, automation, containerization, and deployment orchestration in a realistic software engineering environment.

The final implementation proves that application development and operational efficiency can be combined effectively. It provides a solid foundation for future expansion into larger-scale, production-oriented rental systems and further DevOps integration.